

Model - algorytm otrzymuje kolejne fragmenty wejścia i po każdym musi wypisać odpowiadający fragment wyjścia

Def Algorytm A jest ściśle R -konkurencyjny, gdy

$\forall$ wejścia σ koszt A na $\sigma \leq R \cdot (\text{koszt optymalnego algorytmu offline na } \sigma)$

czyli koszt optymalnego algorytmu, który zna przyszłość

Problem wypożyczania nart (Ski center)

→ koszt wypożyczenia nart 1/dzień

→ koszt zakupu nart B

→ po t dniach psuje się pogoda - wyjeżdżamy

Nie znamy t . W każdym dniu podejmujemy decyzję, czy wypożyczymy (wtedy kolejnego dnia też staniemy przed tą decyzją), czy kupujemy (wtedy wypożyczać się nie przejmujemy).

① Kup pierwszego dnia $\leftrightarrow$ przeciwnie można po 1 dniu kończy się jedna pogoda

$$\left. \begin{array}{l} ALG = B \\ T = 1 \\ OPT = 1 \end{array} \right\} \Rightarrow \text{konkurencyjność to co najmniej } B$$

② Zawsze wynajmujemy $\leftrightarrow$ w najgorszym wypadku pogoda nigdy się nie psuje

$$\left. \begin{array}{l} ALG = \infty \\ T = \infty \\ OPT = B \end{array} \right\} \text{ Alg. nie jest konkurencyjny}$$

③ Kupić po $\frac{B}{2}$ dniach (po zakupie się psuje pogoda)

$$\begin{array}{l} T = \frac{B}{2} + 1 \\ ALG = \frac{B}{2} + B = \frac{3}{2}B \\ OPT = \frac{B}{2} + 1 \end{array}$$

④ Kupić po $B-1$ dniach

$$ALG = \begin{cases} T & T < B \\ B-1 + B = 2B-1 & T \geq B \end{cases}$$

$$OPT = \begin{cases} T & T < B \text{ (gdybyśmy kupili to koszt byłby } B > T) \\ B & B \geq T \end{cases}$$

$$\forall T \text{ koszt}(ALG) \geq \frac{2B-1}{B} \cdot \text{koszt}(OPT)$$

Alg ④ jest ściśle $(2 - \frac{1}{B})$ -konkurencyjny.

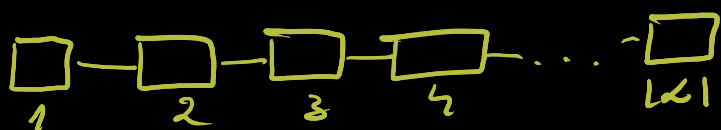
Tw Ścisła konkurencyjność dowolnego deterministycznego algorytmu jest $\geq 2 - \frac{1}{B}$

- $ALG_j =$ "wynajmuj j dni, potem kup" $\{ALG_j : j \in \mathbb{N} \cup \{\infty\}\}$
- $T_j = j+1$ (najgorszy dzień zepsucia się pogody dla ALG_j)
- $\text{cost}(ALG_j) = j + B$
- $\text{cost}(OPT) = \min\{j+1, B\}$

Po elementarnych rachunkach dostajemy:

$$\text{koszt}(ALG_j) \geq (2 - \frac{1}{B}) \cdot \text{koszt}(OPT)$$

Problem reorganizacji listy (list update)



- Zapytania o elementy listy r_t - zapytanie + charł. t .
- Koszt algorytmu to pozycja r_t na liście
- Po zapytaniu można bezpłatnie przenieść r_t w kierunku początku listy
- Można zmienić kolejność dowolnej pary sąsiednich elementów kosztem 1.

Def Algorytm A jest ~~ście~~ R-kongurencyjny, gdy

$\exists \alpha \forall$ wejścia σ koszt A na $\sigma \leq R \cdot (\text{koszt optymalnego algorytmu offline na } \sigma) + \alpha$

$\uparrow$ niezależne od liczby zapytań

$\downarrow$ czyli koszt optymalnego algorytmu, który zna przyszłość

- ① MoveToHalf przesun' r_t o połowę jego pozycji
- ② MoveToFront przesun' r_t na początek
- ③ Transpose przesun' o 1 w lewo
- ④ FrequencyCount - ustawij elementy porządkasz od tego który był dotąd zidentyfikowany najczęściej



Pesymistycznie: zapytania o y i x na przemian
 Opt dla takiego inputu przeniesie x, y na początek
 Algorytm jest $\Omega(1)$ -kongurencyjny
 ④ też jest $\Omega(1)$ -kongurencyjny :-)

Tw ② jest 2-kongurencyjny.

Potencjał Φ = liczba inwersji między permutacjami: ③
 oraz $\Phi \nabla r_t$.

Lemat 1 $\forall r_t$ $\underbrace{\text{koszt}(\text{MTF}, r_t)}_{\text{koszt MTF na zidentyfikowanie } r_t} + \underbrace{\Delta \Phi_t}_{\text{różnica między potencjałem po t krokach i t-1 krokach}} \leq 2 \cdot \text{koszt}(\text{OPT}, r_t)$

Jeżeli Lemat 1 zachodzi, dostajemy:

$$\text{koszt}(\text{MTF}) = \sum_{r_t \in \sigma} [\text{koszt}(\text{MTF}, r_t) + \Delta \Phi_t] \leq 2 \cdot \sum \text{koszt}(\text{OPT}, r_t)$$

$$\text{koszt}(\text{MTF}) \leq \text{koszt}(\text{MTF}) + \underbrace{\Phi_{|\sigma|}}_{\substack{\uparrow \\ 0}} - \underbrace{\Phi_0}_{\substack{\uparrow \\ 0}}$$

Reszta potencjałów skróci się teleskopowo

bo na początku OPT i ALG mają taką samą permutację listy

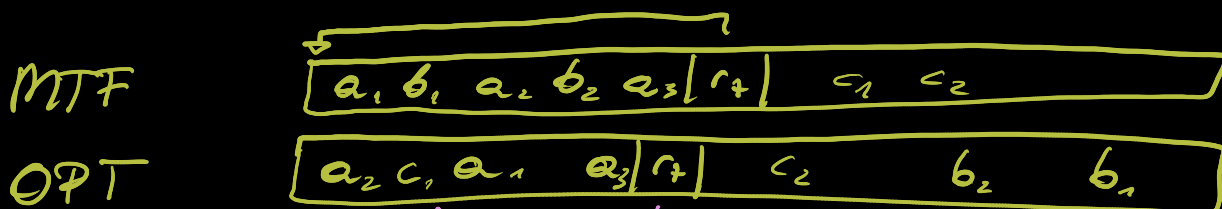
Uwaga

$$\text{koszt}(\text{OPT}) = \sum \text{koszt}(\text{OPT}, r_i) + \frac{\text{swap}(\text{OPT})}{2}$$

nie obchodzi nas, bo algorytm MTF
nie robi swapów i za nie nie płaci

$$\hookrightarrow \Delta \Phi \leq 1 = \text{koszt swapu}$$

d-d lematu 1.



Definiujemy typy elementów:

A - el. przed r_1 w MTF oraz w OPT

B - el. przed r_1 w MTF, ale za r_1 w OPT

C - pozostałe

$$\text{MTF} = |A| + |B| + 1$$

$$\text{OPT} \geq |A| + 1$$

$$\Delta \Phi \leq |A| - |B|$$

$$\text{MTF} + \Delta \Phi \leq 2|A| + |B| - |B| + 1$$

$$\leq 2(|A| + 1) \leq 2 \cdot \text{OPT}$$

